

Desenvolvimento para Dispositivos Móveis

Introdução a Dart

Universidade Federal do Rio Grande do Norte

Introdução

- Breve visão geral do Dart.
- Ano de lançamento: 2011.
- Desenvolvido pela Google.
- Utilizado principalmente com o framework Flutter.
- Flexível com tipagem opcional.
- Recursos poderosos para desenvolvimento de aplicativos móveis e web.

Características Principais

- Tipagem estática opcional.
- Coleta de lixo integrada.
- Suporte a operações assíncronas e concorrência.
- Ideal para desenvolvimento móvel com Flutter.
- Bibliotecas poderosas com diversas funcionalidades já disponíveis.
- Ferramentas de desenvolvimento e integração com IDEs populares.
- Comunidade em crescimento.

Versatilidade

- Multiplataforma: móvel, web e desktop.
- Evolução contínua e suporte da comunidade.
- Recurso para criação de aplicativos modernos e de alto desempenho.
- <https://dartpad.dev/>
- <https://dart-tutorial.com/introduction-and-basics/>
- <https://dart.dev/language>

HelloWorld

Hello World

A função `print(String)` é utilizada para imprimir no console

```
void main() {  
    print("Hello World");  
}
```

HelloWorld

```
void main() {  
    print("Bom dia!: ");  
    print("Hello, world!!!");  
    print ("Taniro " + "Aula");  
}
```

```
Bom dia!:  
Hello, world!!!  
Taniro Aula
```

Strings

- Em Dart uma String contém uma sequência de unidades de código UTF-16.
- Usa-se tanto aspas simples quanto aspas duplas para criar uma string.
- É possível inserir o valor de uma expressão dentro de uma string usando `${expressão}`.
- Se a expressão for um identificador, pode-se omitir as `{}`.
- Para criar uma string com várias linhas, usa-se três aspas com aspas simples ou duplas.

```
"Bom dia $var"
```

Strings

```
void main() {  
    var nome = "Jose";  
    print("Bom dia!: ");  
    print("Hello" + "world!!!");  
    print ("$nome Aula");  
    print ("${nome.length} caracteres");  
    print("""Boa tarde  
    para  
    todo mundo""");  
}
```

```
Bom dia!:  
Helloworld!!!  
Jose Aula  
4 caracteres  
Boa tarde  
    para  
    todo mundo
```

String

- É possível acessar propriedades das variáveis adicionadas a um String
- Para isso precisamos colocar a variável entre chaves conforme exemplo abaixo.
- A propriedades poderão ser acessadas a partir o operador ponto (.)

```
“Bom dia ${var.length}”
```

Variáveis

- Exemplo de uma variável e sua inicialização:

```
var atividade = 'Aula';
```

- Variáveis armazenam referências. A variável chamada **atividade** contém uma referência a uma String com o valor “aula”.
- O tipo da variável **atividade** é inferido como sendo String, mas é possível alterar esse tipo especificando-o.
- Se um objeto não estiver restrito a um único tipo, é possível especificar como do tipo Object (ou dynamic se necessário).

```
String atividade = 'Aula';
```

```
Object atividade = 'Aula';
```

```
dynamic atividade = 'Aula';
```

Null safety

- Null safety evita um erro que resulta do acesso não intencional das variáveis definidas como null.
- Erro de dereferência nulo.
 - Ocorre quando uma propriedade ou um método são invocados em uma expressão que é avaliada como null.
- O compilador Dart detecta esses erros em potencial no momento da compilação.

Null safety

- Quando um **tipo** para uma variável, parâmetro ou outro componente relevante é especificado, podemos controlar se o tipo permite valores nulos.
- Para **habilitar a nulidade**, adiciona-se uma interrogação `<?>` ao final da declaração de tipo.

```
String? name // Nullable type. Can be `null` or string.
```

```
String name // Non-nullable type. Cannot be `null` but can be string.
```

Null safety

- Devemos **inicializar as variáveis** antes de usá-las.
- **Variáveis nulas** por padrão são definidas como nulas, portanto, **são inicializadas por padrão**.
- Dart não define valores iniciais para tipos não nulos. Isso o força o desenvolvedor a definir um valor inicial.
- Não é possível acessar propriedades ou chamar métodos em uma expressão com um tipo null.

Null safety

- O conceito de *null safety* no Dart é uma característica que ajuda a evitar erros relacionados a valores nulos, garantindo que variáveis e propriedades não sejam nulas a menos que sejam explicitamente permitidas a ser.
- O **Sound null safety** transforma potenciais erros em tempo de execução em erros de **análise em tempo de desenvolvimento**.
- O null safety sinaliza uma variável não nula quando ela foi:
 - Não inicializada com um valor não nulo.
 - Atribuída um valor nulo.
- Isso melhora a segurança do código e reduz o risco de exceções em tempo de execução.

```

void main() {
// Exemplo sem null safety
String? nome; // nome pode ser nulo
nome = "João"; // atribuindo um valor não nulo
print(nome.length); // isto funciona

nome = null; // atribuindo nulo
// print(nome.length); // isto geraria um erro em tempo de execução

// Exemplo com null safety
String nomeSeguro = "Maria"; // nomeSeguro não pode ser nulo
print(nomeSeguro.length); // isto funciona

// nomeSeguro = null; // isto geraria um erro de compilação

// Para lidar com valores nulos, você pode usar a checagem nula
nomeSeguro = "Ana";
print(nomeSeguro.length); // isto funciona

// Usando o operador `?` para acessar propriedades de um valor que pode ser nulo
String? nomeOpcional = "Carlos";
print(nomeOpcional?.length); // isto funciona, imprime 6

nomeOpcional = null;
print(nomeOpcional?.length); // isto não gera erro, imprime null

// Usando o operador `!` para afirmar que um valor não é nulo
String? nomeNaoNulo = "Pedro";
print(nomeNaoNulo!.length); // isto funciona, imprime 5

// Se você tentar usar `!` em um valor nulo, o código lançará uma exceção
// String? nomeNulo = null;
// print(nomeNulo!.length); // Isto geraria um erro de execução
}

```

Introdução

Strings Templates

Variáveis

Null Safety

Late, Const, Final

Operadores

Funções

Null Safety

- **Variáveis com possibilidade de nulo (String?):** Variável declarada com um ?, está permitindo que ela possa ser nula.
- **Variáveis não nulas (String):** Variável sem ?, você garante que ela nunca será nula. Isso previne atribuições nulas e forçará você a sempre atribuir um valor não nulo à variável.
- **Operador ?. (safe navigation operator):** Esse operador é usado para acessar propriedades de um objeto que pode ser nulo sem lançar uma exceção. Se o objeto for nulo, o resultado da expressão será nulo.
- **Operador ! (null assertion operator):** Esse operador é usado para afirmar que um valor não é nulo. Se o valor for nulo, ele lançará uma exceção `NullPointerException`.

Variáveis late

- O modificador 'late' tem dois casos de uso:
 - Declarar uma variável não nula que é inicializada após a sua declaração.
 - Inicializar uma variável de forma lazy.
- Frequentemente, a análise de fluxo de controle do Dart pode detectar quando uma variável não-null é definida com um valor não null antes de ser usada, mas às vezes a análise falha.
- Se uma variável for definida antes de ser usada, mas o Dart discorda, é possível corrigir o erro marcando a variável como 'late'

```
late String description;  
void main() {  
    description = 'Feijoadada!';  
    print(description);  
}
```

final e const

- Se uma variável não terá seu valor alterado, use-se 'final' ou 'const'.
- Uma variável 'final' pode ser definida apenas uma vez.
- Uma variável 'const' é uma constante em tempo de compilação.
 - Variáveis 'const' são implicitamente 'final'.

```
final name = 'Bob'; // Without a type annotation  
final String nickname = 'Bobby';
```

final e const

- Usa-se 'const' para variáveis sejam constantes em tempo de compilação.
- Se a variável 'const' estiver no nível da classe, ela deve ser marcada como 'static const'.

```
const bar = 1000000; // Unit of pressure (dynes/cm2)
const double atm = 1.01325 * bar; // Standard atmosphere
```

Operadores

- Dart possui a maioria dos operadores conhecidos de outras linguagens como Java
- Os próximos slides detalham outros operadores específicos da linguagem.

Condicionais ? e ??

- Dart possui dois operadores que permitem avaliar concisamente expressões que poderia exigir utilizar if/else:

`condition ? expr1 : expr2`

- Se condição é verdade, avalia `expr1` (e retorna seu valor); caso contrário, avalia e retorna o valor de `expr2`.

`expr1 ?? expr2`

- Se `expr1` não é nulo, retorna seu valor; caso contrário, avalia e retorna o valor de `expr2`.

```
String playerName(String? name) => name ?? 'Guest';
```

Notação em Cascata

- Cascatas (... ?..) permitem fazer uma sequência de operações no mesmo objeto.
- Além de acessar os membros da instância, é possível chamar métodos de instância nesse mesmo objeto.
- Isso geralmente salva a etapa de criar uma variável temporária e permite escrever mais código fluido.

```
var paint = Paint()  
  ..color = Colors.black  
  ..strokeCap = StrokeCap.round  
  ..strokeWidth = 5.0;
```

```
var paint = Paint();  
paint.color = Colors.black;  
paint.strokeCap = StrokeCap.round;  
paint.strokeWidth = 5.0;
```

Conversão de Tipo

- Para realizar a operação de cast em Dart, converter um tipo em outro, é utilizado operador 'as'.
- O operador 'as' pode lançar uma exceção do tipo **TypeError** caso a conversão não possa ser feita.
- Operador 'is' pode ser utilizado para verificar se uma variável é de um tipo.

Variáveis dynamic

- Uma variável inicializada com a palavra-chave 'dynamic' possui tipo dynamic.
- A variável declarada com esse tipo de dado pode armazenar implicitamente qualquer valor durante a execução do programa.

```
void main() {  
    dynamic pessoa = "Taniro";  
    print(pessoa);  
    print(pessoa.runtimeType);  
    pessoa = 57;  
    print(pessoa.runtimeType);  
}
```

Taniro
String
int

Funções

- Dart é uma linguagem verdadeiramente orientada a objetos, então até mesmo funções são objetos e têm um tipo, 'Function'.
- Isso significa que as funções podem ser atribuídas a variáveis ou passadas como argumentos para outras funções.
- A declaração de funções segue a seguinte sintaxe:

```
returntype functionName(parameter1,parameter2, ...){  
    // function body  
}
```

Funções

- Funções sem retorno possuem o tipo void.

```
void imprime(String texto) {  
    print(texto)  
}
```

Funções

- Sempre que uma função tiver apenas uma linha, não será preciso abrir e fechar as chaves{}
- Basta usar o operador => e escrever tudo em uma única linha.

```
void imprime(String palavra) => print(palavra);  
int calcula(int a, int b)  => a + b;
```

Funções

- Uma função pode ter **qualquer número de parâmetros posicionais obrigatórios**.
- Estes podem ser **seguidos** por **parâmetros nomeados** ou por **parâmetros posicionais opcionais** (mas não ambos).
- Parâmetros nomeados são opcionais, a menos que sejam explicitamente marcados como obrigatórios com a palavra-chave `required`.
- Ao definir uma função `{param1, param2, ...}` é usado para especificar parâmetros nomeados.
- Se for fornecido um valor padrão ou um parâmetro nomeado marcado como obrigatório, seus tipos devem ser `null`, pois o valor padrão deles será `null`.

Funções

● Exemplo:

```
/// Sets the [bold] and [hidden] flags ...  
void enableFlags({bool? bold, bool? hidden}) {...}
```

● Chamada:

```
enableFlags(bold: true, hidden: false);
```

Funções

- Para definir um valor padrão para um parâmetro nomeado além de nulo, é usado o sinal de igual (=) para especificar um valor padrão.
- O valor especificado deve ser uma constante em tempo de compilação.

```
/// Sets the [bold] and [hidden] flags ...  
void enableFlags({bool bold = false, bool hidden = false}) {...}  
// bold will be true; hidden will be false.  
enableFlags(bold: true);
```

Funções

- Se, em vez disso, for desejado que um parâmetro nomeado seja obrigatório, exigindo que os chamadores forneçam um valor para o parâmetro, anote-os com 'required'

```
const Scrollbar({super.key, required Widget child});
```

Funções

```
void main() {  
    print (calcula(40, valorb: 22));  
}  
  
int calcula (int valorc, { int valora = 7, required int valorb}){  
    return valora + valorb + valorc;  
}
```

Funções como First-class objects

- É possível passar funções como parâmetro para outras funções.

```
void printElement(int element) {  
    print(element);  
}  
var list = [1, 2, 3];  
  
list.forEach(printElement);
```

Funções anônimas

- A maioria das funções tem nomes, como `main()` ou `printElement()`.
- É possível criar uma função sem nome, chamada de função anônima, às vezes chamada de lambda ou closure.
- É possível atribuir uma função anônima a uma variável, para, por exemplo, adicioná-la ou removê-la de uma coleção.

Arrow Function

- Dart possui uma sintaxe especial para o corpo da função, que consiste em apenas uma linha.
- A função de seta (arrow function) é representada pelo símbolo `=>`.
- Essa é uma sintaxe abreviada para qualquer função que tenha apenas uma expressão.

Funções anônimas

```
void imprime(int palavra) => print(palavra);  
var escreve = (e) => print(e);  
var soma = (int a, int b) {  
    return a + b;  
};
```

```
void main() {  
    print(escreve.runtimeType);  
    print(soma.runtimeType);  
    list.forEach( (e) => print(e));  
}
```

Funções anônimas

```
void exec(int i, Function f) {  
    print(f(i));  
}  
  
int plusOne(int i) => i + 1;  
int double(int i) => i * 2;  
  
void main() {  
  
    exec(10, plusOne);  
    exec(10, double);  
  
    exec(10, (x) => x + 1); //plusOne  
    exec(10, (x) => x * 2); //double  
}
```

Função Closure

- Um closure é um objeto de função que tem acesso a variáveis em seu escopo léxico, mesmo quando a função é usada fora de seu escopo original.
- As funções podem fechar (close) sobre (over) variáveis definidas em escopos circundantes.

```
Function makeAdder(int addBy) {  
    return (int i) => addBy + i;  
}  
void main() {  
    // Create a function that adds 2.  
    var add2 = makeAdder(2);  
    // Create a function that adds 4.  
    var add4 = makeAdder(4);  
    assert(add2(3) == 5); assert(add4(3) == 7);  
}
```

Coleções

- Dart possui três tipos de coleções:
 - List
 - Map
 - Set

List

- Caso seja necessário armazenar múltiplos valores na mesma variável, pode usar uma List.
- Em Dart, uma List é semelhante a **Arrays** em outras linguagens de programação.
 - Por exemplo, para armazenar os nomes de várias pessoas é possível usar uma List. Uma List é representada por colchetes [].

```
// Integer List
```

```
List<int> ages = [10, 30, 23];
```

```
// String List
```

```
List<String> names = ["Raj", "John", "Rocky"];
```

```
// Mixed List
```

```
var mixed = [10, "John", 18.8];
```

List

- Existem dois tipos de lists:
 - Tamanho fixo, e
 - Tamanho variável
- As Listas de tamanho fixo são definidas com um tamanho especificado.
 - Não é possível alterar o tamanho delas durante a execução.

```
void main() {  
    var list = List<int>.filled(5,0);  
    print(list);  
}
```

List

- Existem dois tipos de lists:
 - Tamanho fixo, e
 - Tamanho variável
- Uma List definida sem um tamanho especificado é chamada de Growable List.
 - O tamanho da Growable List pode ser alterado durante a execução.

```
void main() {  
    var list1 = [210,21,22,33,44,55];  
    print(list1);  
}
```

Sets

- Um Set é uma coleção de itens únicos.
- Não é possível armazenar valores duplicados em um Set.
- O Set é não ordenado, o que pode torná-lo mais eficiente do que as Listas ao lidar com uma grande quantidade de dados.
- Os Sets são úteis quando é necessário armazenar valores únicos sem considerar a ordem de entrada.
- Os Sets são representados por chaves {}.

Sets

```
void main() {  
    // declaring fruits as Set  
    Set<String> fruits = {"Apple", "Orange", "Mango", "Banana"};  
  
    // using different properties of Set  
    print("First Value is ${fruits.first}");  
    print("Last Value is ${fruits.last}");  
    print("Is fruits empty? ${fruits.isEmpty}");  
    print("Is fruits not empty? ${fruits.isNotEmpty}");  
    print("The length of fruits is ${fruits.length}");  
}
```

Maps

- Em um Map, os dados são armazenados como pares de chave e valor.
- Em um Map, cada chave deve ser única.
- Os Maps são semelhantes aos HashMaps e Dicionários em outras linguagens de programação.

```
void main(){  
  Map<String, String> countryCapital = {  
    'USA': 'Washington, D.C.',  
    'India': 'New Delhi',  
    'China': 'Beijing'  
  };  
  print(countryCapital["USA"]);  
}
```

Where

- É possível usar o where em lists, sets e maps para filtrar itens específicos.
- O Where retorna uma nova lista contendo todos os elementos que satisfazem a condição.
- Isso também é chamado de filtro where em Dart.

```
void main() {  
    List<int> numbers = [2, 4, 6, 8, 10, 11, 12, 13, 14];  
  
    List<int> oddNumbers = numbers.where((number) => number.isOdd).toList();  
    print(oddNumbers);  
}
```

Programação Orientada a Objetos

- Classes
- Construtores
- Getters, Setters
- Encapsulação
- Herança
- Polimorfismo
- Interfaces
- Classes Abstratas

Classes

- Para declarar uma classe usa-se a palavra-chave `class`, seguida pelo nome da classe e chaves `{}`.
- É uma boa prática escrever o nome da classe em `PascalCase`.
 - Por exemplo, `Empregado`, `Escola`, `Curso`, etc.

```
class ClassName {  
    // properties or fields  
    // methods or functions  
}
```

Classes

- O corpo da classe consiste em propriedades e funções.
- Propriedades são usadas para armazenar dados e também são conhecidas como campos ou atributos.
- Funções são usadas para realizar operações e também são conhecidas como métodos.

Classes

```
class Animal {  
    String? nome;  
    int? numeroDePernas;  
    String? corDoPelo;  
  
    void display() {  
        print("Nome: $nome.");  
        print("Número de pernas: $numeroDePernas.");  
        print("Cor do Pelo: $corDoPelo.");  
    }  
}  
  
void main() {  
    var cachorro = Animal()  
        ..corDoPelo = "Preto"  
        ..nome = "Rex"  
        ..numeroDePernas = 4  
        ..display();  
}
```

Construtor

- Um construtor é um método especial usado para inicializar um objeto.
- Ele é chamado automaticamente quando um objeto é criado e pode ser usado para definir os valores iniciais das propriedades do objeto.

```
class ClassName {  
    ClassName() {  
        // body of the constructor  
    }  
}
```

```
Animal(String nome, int numeroDePernas, String corDoPelo){  
    this.nome = nome;  
    this.numeroDePernas = numeroDePernas;  
    this.corDoPelo = corDoPelo;  
}
```

Construtor em única linha

```
class Animal {  
    String nome;  
    int numeroDePernas;  
    String corDoPelo;  
  
    Animal(this.nome, this.numeroDePernas, this.corDoPelo);  
  
    void display() {  
        print("Nome: $nome.");  
        print("Número de pernas: $numeroDePernas.");  
        print("Cor do Pelo: $corDoPelo.");  
    }  
}  
  
void main() {  
    var cachorro = Animal("Rex", 4, "Preto")  
        ..display();  
}c
```

Construtor com parâmetros opcionais

```
class Animal {  
    String nome;  
    int numeroDePernas;  
    String? corDoPelo;  
  
    Animal(this.nome, [this.numeroDePernas = 0, this.corDoPelo]);  
  
    void display() {  
        print("Nome: $nome.");  
        print("Número de pernas: $numeroDePernas.");  
        print("Cor do Pelo: $corDoPelo.");  
    }  
}  
  
void main() {  
    var cachorro = Animal("Rex", 4, "Preto")  
        ..display();  
  
    var caracol = Animal("Rapidex")  
        ..display();  
}
```

Construtor com parâmetros nomeados

```
class Animal {  
    String? nome;  
    int? numeroDePernas;  
    String? corDoPelo;  
  
    Animal({this.nome, this.numeroDePernas, this.corDoPelo});  
  
    void display() {  
        print("Nome: $nome.");  
        print("Número de pernas: $numeroDePernas.");  
        print("Cor do Pelo: $corDoPelo.");  
    }  
}  
  
void main() {  
    var cachorro = Animal(corDoPelo: "Preto", numeroDePernas: 4, nome : "Rex")  
        ..display();  
}
```

Construtor nomeado

- Na maioria das linguagens de programação, como Java, C++, C#, etc., é possível criar múltiplos construtores com o mesmo nome.
- No entanto, em Dart, isso não é possível diretamente.
- No entanto, é possível criar múltiplos construtores com o mesmo nome usando construtores nomeados.
- Isso permite existam várias maneiras de inicializar um objeto da classe, cada uma com um conjunto diferente de parâmetros.

Construtor nomeado

```
class Animal {  
    String? nome;  
    int? numeroDePernas;  
    String? corDoPelo;  
  
    Animal({this.nome, this.numeroDePernas, this.corDoPelo});  
  
    Animal.semPelo(this.numeroDePernas, this.nome);  
  
    void display() {  
        print("Nome: $nome.");  
        print("Número de pernas: $numeroDePernas.");  
        print("Cor do Pelo: $corDoPelo.");  
    }  
}  
  
void main() {  
    var cachorro = Animal(corDoPelo: "Preto", numeroDePernas: 4, nome : "Rex")  
        ..display();  
    var tartatura = Animal.semPelo(4, "Turtie")  
        ..display();  
}
```

Construtor constante

- Um construtor constante é um construtor que cria um objeto constante.
- Um objeto constante é um objeto cujo valor não pode ser alterado após a criação.
- Um construtor constante é declarado usando a palavra-chave `const`.
- Útil quando para garantir que um objeto não seja modificado após a criação e que ele seja imutável.

Construtor constante

```
class Animal {  
    final String nome;  
    final int numeroDePernas;  
    final String corDoPelo;  
  
    const Animal(this.nome, this.numeroDePernas, this.corDoPelo);  
  
    Animal.semPelo(this.numeroDePernas, this.nome, [this.corDoPelo="NA"]);  
  
    void display() {  
        print("Nome: $nome.");  
        print("Número de pernas: $numeroDePernas.");  
        print("Cor do Pelo: $corDoPelo.");  
    }  
}  
  
void main() {  
    var cachorro = const Animal("Rex", 4, "Preto")  
        ..display();  
  
    var tartatura = Animal.semPelo(4, "Turtie")  
        ..display();  
}
```

Encapsulamento

- Encapsulamento significa ocultar dados dentro de uma biblioteca, impedindo que fatores externos tenham acesso a eles.
- Isso ajuda a controlar o programa e evita que ele se torne muito complicado, uma vez que fornece uma camada de proteção para os dados, permitindo que seja definido como esses dados podem ser acessados e modificados.

Encapsulamento

- O Encapsulamento pode ser alcançada por meio das seguintes práticas:
 - Declarar as propriedades da classe como privadas usando um underscore `_` no início do nome da propriedade. Isso indica que essas propriedades não devem ser acessadas diretamente de fora da classe.
 - Fornecer métodos públicos "getter" e "setter" para acessar e atualizar o valor das propriedades privadas.

Getter e Setter

```
class Employee {  
    // Private property  
    var _name;  
  
    // Getter method to access private property _name  
    String getName() {  
        return _name;  
    }  
  
    // Setter method to update private property _name  
    void setName(String name) {  
        this._name = name;  
    }  
}  
  
void main() {  
    var employee = Employee();  
    employee.setName("Jack");  
    print(employee.getName());  
}
```

Getter e Setter

- A privacidade em Dart ocorre no nível da biblioteca, em vez de no nível da classe.
- Isso significa que a visibilidade de membros de classe (como propriedades e métodos) é restrita ao escopo da biblioteca em que estão definidos.
- Membros de classe marcados com um underscore `_` no início de seus nomes são considerados privados e só podem ser acessados dentro da biblioteca em que são definidos.

Getter e Setter

- Os getters e setters proporcionam acesso explícito de leitura e escrita às propriedades de um objeto. Em Dart, as palavras-chave `get` e `set` são usadas para criar getters e setters.
- Um getter lê o valor de uma propriedade e atua como um acessador, permitindo obter o valor de uma propriedade de um objeto.
- Um setter atualiza o valor de uma propriedade e atua como um mutador, definir ou modificar o valor de uma propriedade de um objeto.
- Isso oferece controle sobre como os dados são lidos e escritos, permitindo a implementação de lógica personalizada ao acessar ou modificar propriedades de objetos.

Getter e Setter

```
class Student {  
    String? _firstName;  
    String? _lastName;  
    int? _age;  
  
    String get fullName => this._firstName! + " " + this._lastName!  
  
    int get age => this._age!  
  
    set firstName(String firstName) => this._firstName = firstName;  
    set lastName(String lastName) => this._lastName = lastName;  
    set age(int age) {  
        if (age < 0) {  
            throw new Exception("Age can't be less than 0");  
        }  
        this._age = age;  
    }  
}  
  
void main() {  
    Student st = new Student();  
    st.firstName = "John";  
    st.lastName = "Doe";  
    st.age = 20;  
    print("Full Name: ${st.fullName}");  
    print("Age: ${st.age}");  
}
```

Herança

- A herança é o compartilhamento de comportamento entre duas classes.
- Ela permite que definir uma classe que estende a funcionalidade de outra classe.
- A palavra-chave **extends** é usada para herdar de uma classe pai (ou superclasse) em Dart.
- Isso significa que a classe filha (ou subclasse) herda os membros (propriedades e métodos) da classe pai, permitindo que reutilizar e estender o comportamento da classe pai na classe filha. Isso é um conceito fundamental na programação orientada a objetos e ajuda a promover a reutilização de código.

Herança

```
class Laptop {  
    // Method  
    void show() {  
        print("Laptop show method");  
    }  
}  
  
class MacBook extends Laptop {  
    void show() {  
        super.show(); // Calling the show method of the parent class  
        print("MacBook show method");  
    }  
}  
  
void main() {  
    // Creating an object of the MacBook class  
    MacBook macbook = MacBook();  
    macbook.show();  
}
```

Polimorfismo

- Polimorfismo é a capacidade de um objeto assumir várias formas, o que significa que um objeto pode se comportar de maneira diferente dependendo do contexto em que é usado.
- Isso permite que você escreva código mais flexível e genérico, que pode trabalhar com diferentes tipos de objetos de maneira consistente.

```
class ParentClass{  
    void functionName(){  
        }  
    }  
class ChildClass extends ParentClass{  
    @override  
    void functionName(){  
        }  
    }
```

Referências

- <https://dart.dev/>
- <https://dart-tutorial.com/>